MOBL-02: iOS SDK Setup (Swift & SwiftUI)

> **Series:** MOBL — Mobile Monitoring | **Notebook:** 2 of 12 | **Created:** February 2026 | **Last Updated:** 11/11/2026

Overview

This notebook walks through setting up the **Dynatrace Mobile RUM SDK** for iOS applications using **Swift Package Manager (SPM)** or **CocoaPods**. You will learn how to create a mobile app configuration in Dynatrace, install and configure the SDK, instrument both UIKit and SwiftUI views, and verify that session and action data is flowing into your Dynatrace environment.

The Dynatrace iOS SDK provides:
- **Automatic user-action detection** for UIKit view controllers, navigation, and network requests
- **Crash reporting** with symbolicated stack traces
- **SwiftUI instrumentation** at build time by the Dynatrace SwiftUI instrumentor (`DTSwiftInstrumentor`)
- **Manual action and event APIs** for custom business logic

Table of Contents

1. Creating a Mobile App in Dynatrace
2. Installing the SDK
3. Configuring Info.plist
4. Auto-Instrumentation (UIKit)
5. SwiftUI Integration
6. Manual Startup Configuration

7. Verifying Data in Dynatrace

Prerequisites

Requirement	Details
Xcode	16.0 or later
iOS Deployment Target	iOS 15.0 or later (tvOS 15.0+). From April 2027 Dynatrace stops supporting iOS 15 and 16; the minimum becomes iOS 17
Dynatrace Environment	SaaS with Grail, with a mobile frontend created in Experience Vitals (RUM Classic frontends: RUM turned on for the frontend; Section 1)
Mobile App Configuration	A mobile app created in Dynatrace (or you will create one in Section 1)
Language	Swift 5.7+
Permissions	`storage:user.events:read`, `storage:user.sessions:read` (mobile RUM on Grail), `storage:smartscape:read` (app inventory)

> **Sources:** Initial setup for iOS frontends (DT docs) — *"iOS 15.0+ tvOS 15.0+ Xcode 16.0+"*, *"Starting April 2027 Dynatrace will stop supporting iOS 15 and iOS 16."*

1. Creating a Mobile App in Dynatrace

Before integrating the SDK into your iOS project, you need to create a **mobile application configuration** in Dynatrace. This generates two critical values:

- **Application ID** (`DTXApplicationID`) -- uniquely identifies your app in Dynatrace
- **Beacon URL** (`DTXBeaconURL`) -- the endpoint where the SDK sends telemetry data

Steps

1. **Create the frontend:** open **Experience Vitals**, select **Add Frontend**, and follow the Frontend creation wizard with **iOS** as the platform. The wizard gives you the

Application ID and **Beacon URL** — copy both.

2. Optionally enable **Crash reporting**, **Session replay**, or **User tagging** from the frontend settings.

> **Upgrading a RUM Classic iOS frontend?** Turn RUM on for it: **Experience Vitals > Overview > Mobile** → select the frontend → **Settings** → under **Enablement and cost control**, turn on **RUM**. For all mobile frontends at once: **Settings > Collect and capture > Real User Monitoring > Enablement and cost control > Mobile** → **RUM**. The `user.events` / `user.sessions` queries in this series read the data RUM sends to Grail.
>
> **Tip:** The instrumentation wizard in Experience Vitals shows your Application ID and Beacon URL at any time. (RUM Classic tenants find them under **Mobile > Your App > Settings > General**.)
>
> **Sources:** Initial setup for iOS frontends (DT docs) — *"Select Add Frontend in the upper-right corner. Follow the steps in the Frontend creation wizard . Select iOS as your platform."*; Enable RUM for iOS frontends (DT docs) — *"If you already monitor your iOS frontends with RUM Classic, upgrading to RUM requires only a configuration change."*, *"Under Enablement and cost control , turn on RUM ."*

2. Installing the SDK

The Dynatrace iOS SDK can be installed via **Swift Package Manager (SPM)** or **CocoaPods**. Choose the method that fits your project's dependency management.

Option A: Swift Package Manager (Recommended)

SPM is Apple's native dependency manager and requires no additional tooling.

In Xcode:

1. Go to **File > Add Package Dependencies...**
2. Enter the Dynatrace iOS SPM repository URL.
3. Select the version rule (e.g., **Up to Next Major Version**).
4. Add the `Dynatrace` library to your app target.

```
// Package.swift (if using a Package.swift-based project)
dependencies: [
    .package(url: "https://github.com/Dynatrace/swift-mobile-sdk", from:
"8.0.0")
]
```

The repository is `github.com/Dynatrace/swift-mobile-sdk` (tags follow the agent version, for example `8.347.1`).

Option B: CocoaPods

If your project uses CocoaPods, add the Dynatrace pod to your `Podfile`:

```
# Podfile
platform :ios, '15.0'
use_frameworks!

target 'MyApp' do
  pod 'Dynatrace', '~&gt; 8.0'
end
```

Then run:

```
pod install
```

> **Note:** After installing via CocoaPods, always open the `.xcworkspace` file (not `.xcodeproj`) for subsequent builds.

3. Configuring Info.plist

The simplest way to configure the Dynatrace SDK is through your app's `Info.plist` file. When `DTXAutoStart` is set to `true`, the SDK initializes automatically at app launch.

Add the following keys to your `Info.plist`:

```
&lt;key&gt;DTXApplicationID&lt;/key&gt;
&lt;string&gt;YOUR_APP_ID&lt;/string&gt;
&lt;key&gt;DTXBeaconURL&lt;/key&gt;
&lt;string&gt;YOUR_BEACON_URL&lt;/string&gt;
&lt;key&gt;DTXAutoStart&lt;/key&gt;
&lt;true/&gt;
&lt;key&gt;DTXCrashReportingEnabled&lt;/key&gt;
&lt;true/&gt;
```

Key Reference

Key	Type	Description
`DTXApplicationID`	String	The Application ID from your Dynatrace mobile app configuration
`DTXBeaconURL`	String	The Beacon URL endpoint for telemetry data
`DTXAutoStart`	Boolean	When `true`, the SDK starts automatically at app launch
`DTXCrashReportingEnabled`	Boolean	When `true`, enables crash reporting with symbolicated stack traces
`DTXHybridApplication`	Boolean	Set to `true` if the app uses WKWebView hybrid content
`DTXUserOptIn`	Boolean	When `true`, OneAgent captures no data until the user opts in via `Dynatrace.applyUserPrivacyOptions(...)`
`DTXStartupWithGrailEnabled`	Boolean	When `true`, sends data to Grail from the **first** app start (default `false`)

> **Sources:** Configuration (DT docs) — *"DTXAutoStart Boolean true When true, OneAgent starts automatically when your app launches."*; Initial setup for iOS frontends (DT docs) — *"DTXStartupWithGrailEnabled false Sends data to Grail from the first app start"*.
>
> **Important:** Replace `YOUR_APP_ID` and `YOUR_BEACON_URL` with the actual values from your Dynatrace mobile app configuration (see Section 1).

4. Auto-Instrumentation (UIKit)

When the SDK is properly configured and `DTXAutoStart` is enabled, the Dynatrace agent automatically instruments standard UIKit components without any code changes. This is the primary advantage of the Dynatrace mobile SDK -- **zero-code instrumentation** for common patterns.

What Gets Auto-Detected

Component	What Is Captured
UIViewController lifecycle	`viewDidAppear`, `viewDidDisappear` -- tracked as user actions with the view controller class name
UITableView / UICollectionView taps	Cell selection events including index path and reuse identifier
UINavigationController	Push/pop navigation transitions and associated view controller names
UITabBarController	Tab selection changes
URLSession network requests	HTTP method, URL, status code, response size, and timing for all requests made through `URLSession`
WKWebView	Web request monitoring within hybrid views (requires `DTXHybridApplication` set to `true`)
App lifecycle	App launch, foreground/background transitions, session start/end
Crashes	Unhandled exceptions and signal crashes with symbolicated stack traces

How It Works

The SDK uses method swizzling at runtime to intercept UIKit delegate methods and lifecycle callbacks. This means:

- No subclassing or protocol conformance required
- Works with existing `UIViewController` subclasses automatically
- Network monitoring hooks into the `URLSession` delegate chain

- Crash reporting installs signal and exception handlers

> **Note:** Auto-instrumentation covers most common UIKit patterns. For custom gestures, programmatic transitions, or non-standard networking libraries, use the manual action API (covered in **MOBL-05 §5**).

5. SwiftUI Integration

SwiftUI does not use `UIViewController` in the traditional sense, so the runtime swizzling that instruments UIKit does not reach SwiftUI controls. Dynatrace instruments them **at build time** instead, with the **Dynatrace SwiftUI instrumentor** (`DTSwiftInstrumentor` , OneAgent for iOS 8.249+). During each build the instrumentor adds observation code to your `*.swift` files, notifies OneAgent about UI-element state changes, and reverts the source changes after the build completes. There is no SwiftUI view modifier to add by hand.

Install the instrumentor (Homebrew)

```
brew tap dynatrace/tools
brew trust dynatrace/tools/DTSwiftInstrumentor   # Homebrew 6.0.0+ only; safe
on earlier versions
brew install DTSwiftInstrumentor
```

Then quit Xcode and run:

```
DTSwiftInstrumentor install    # optional: &lt;PROJECT.xcodeproj&gt; --scheme
&lt;SCHEME&gt; --target &lt;TARGET&gt;
```

Without project arguments the tool auto-detects targets and schemes and starts an interactive selection. A manual (ZIP) install path is also documented.

What gets instrumented

OneAgent for iOS	Supported SwiftUI controls
8.249+	`Button` , `Stepper` , `Picker` , `Toggle` , `Slider`

OneAgent for iOS	Supported SwiftUI controls
8.265+	`PasteButton`, `EditButton`, `RenameButton`, `Link`, `ShareLink`, `NavigationLink`, `DatePicker`, `MultiDatePicker`, `ColorPicker`, `TabView`, `List` -- plus closures such as `onTapGesture`, `refreshable`, `sheet`, `popover`, `navigationDestination`
8.269+	`Menu`, `WindowGroup`

Requirements: SwiftUI 2.0+, iOS 14+. Controls can be excluded globally or locally when needed.

When to add manual actions

Use the manual action API only for **business flows the instrumentor cannot see** -- for example, a checkout that spans several screens:

```
import Dynatrace

let checkout = DTXAction.enter(withName: "Checkout flow")
// ... several screens and requests later ...
checkout?.leave()
```

> **Correction (09/28/2026).** Earlier revisions of this section told readers to decorate SwiftUI views with a `.dtAction(name:)` modifier. No such modifier exists in the Dynatrace iOS SDK, so those samples did not compile. SwiftUI controls are instrumented by `DTSwiftInstrumentor`.
>
> **Sources:** Instrument SwiftUI controls (DT docs) — *"Run brew install DTSwiftInstrumentor to install our SwiftUI instrumentor."*, OneAgent SDK for iOS (DT docs).

6. Manual Startup Configuration

If you need more control over when and how the SDK starts (for example, to inject configuration values from a remote config service), set `DTXAutoStart` to `false` in `Info.plist` and start the SDK programmatically — *"Make sure to disable auto-start if you plan on starting the agent manually."* Values passed at startup take precedence over

`Info.plist`. Manual startup runs later than automatic startup, so early lifecycle events such as the application start are not captured until the agent is initialized.

AppDelegate (UIKit Lifecycle)

```
import UIKit
import Dynatrace

class AppDelegate: UIResponder, UIApplicationDelegate {
    func application(
        _ application: UIApplication,
        didFinishLaunchingWithOptions launchOptions:
[UIApplication.LaunchOptionsKey: Any]?
    ) -&gt; Bool {

        Dynatrace.startupWithConfig([
            kDTXApplicationID: "YOUR_APP_ID",
            kDTXBeaconURL: "YOUR_BEACON_URL"
        ])

        return true
    }
}
```

SwiftUI App Lifecycle

For apps using the SwiftUI `App` protocol (no `AppDelegate`):

```
import SwiftUI
import Dynatrace

@main
struct MyApp: App {
    init() {
        Dynatrace.startupWithConfig([
            kDTXApplicationID: "YOUR_APP_ID",
            kDTXBeaconURL: "YOUR_BEACON_URL"
        ])
    }

    var body: some Scene {
        WindowGroup {
            ContentView()
        }
    }
}
```

Deferred Start (User Consent)

If your app requires user opt-in before monitoring:

For consent, the documented mechanism is **user opt-in mode**, not deferring startup: set `DTXUserOptIn` to `true`, and OneAgent captures nothing until you apply the user's choice with `Dynatrace.applyUserPrivacyOptions(...)` (MOBL-09). `DTXAutoStart` is `true` by default — omitting it does **not** defer startup.

> **Important:** Replace `YOUR_APP_ID` and `YOUR_BEACON_URL` with the actual values from your Dynatrace mobile app configuration.
>
> **Sources:** Configuration (DT docs) — *"Make sure to disable auto-start if you plan on starting the agent manually."*, *"Manual startup occurs later, meaning those early events aren't captured until the agent is initialized."*

7. Verifying Data in Dynatrace

After installing and configuring the SDK, launch your iOS app on a device or simulator and interact with a few screens. Data should begin appearing within a few minutes. Without

`DTXStartupWithGrailEnabled`, the very first app start may not reach Grail — launch the app a second time before concluding that nothing arrives.

Where to Check

Location	What to Verify
Experience Vitals > Overview > Mobile > your frontend	Sessions and user actions appear (RUM Classic: **Mobile > Your App**)
Error Inspector	Crash reporting is active (trigger a test crash if needed)
Your frontend's requests view	HTTP calls from URLSession appear
Notebooks / DQL	Query for mobile entities and actions programmatically

The following DQL queries help confirm that data is arriving from your iOS app.

Find iOS Mobile Applications

This query lists mobile application entities that contain "iOS" in their name or tags, confirming your app is registered in Dynatrace.

```
// Find iOS mobile applications
// PREFERRED -- Smartscape. FRONTEND carries both mobile and web apps, so the
// frontend.type filter is what narrows this to mobile.
smartscapeNodes "FRONTEND"
| filter frontend.type == "mobile"
| filter contains(name, "iOS", caseSensitive: false)
    or contains(toString(tags), "iOS", caseSensitive: false)
| fields name, id, id_classic, tags
| sort name asc

// CORRECTION (07/30/2026): a previous revision claimed
dt.entity.mobile_application had no
// Grail Smartscape equivalent. It does -- FRONTEND filtered on frontend.type
== "mobile"
// (dt.smartscape.frontend). SaaS 1.344 (07/27/2026, staged tenant rollout
from 07/29/2026)
// makes Smartscape the primary surface for the Digital Experience apps;
verify it has
// reached your tenant. The classic table below still works and remains a real
fallback.
// FALLBACK (classic surface -- still functional):
// fetch dt.entity.mobile_application
// | filter contains(toString(entity.name), "iOS") or contains(toString(tags),
"iOS")
// | fields entity.name, id, tags
// | sort entity.name asc
```

Verify Beacon Data Arriving

This query checks for recent mobile user actions received from iOS devices in the last hour. Mobile RUM data lands in the Grail `user.events` and `user.sessions` stores; `characteristics.has_user_action` selects user-action events. If rows appear, the SDK is successfully sending telemetry.

```
// Recent mobile user actions on iOS (last hour)
// Mobile RUM is stored in user.events (one record per event, typed by the
// characteristics.has_* flags) and user.sessions -- not in bizevents.
fetch user.events, from:-1h
| filter in(dt.rum.agent.type, {"android", "ios"}) and os.name == "iOS"
| filter characteristics.has_user_action
| summarize action_count = count(), by:{frontend.name,
ui_element.detected_name, interaction.type}
| sort action_count desc
| limit 20
```

Look Up App Entity Details

This query retrieves general details for your mobile app entities, including their lifetime and any assigned tags.

```
// iOS app entity details
// PREFERRED -- Smartscape
smartscapeNodes "FRONTEND"
| filter frontend.type == "mobile"
| fields name, id, id_classic, lifetime, tags
| limit 10

// CORRECTION (07/30/2026): a previous revision claimed
dt.entity.mobile_application had no
// Grail Smartscape equivalent. It does -- FRONTEND filtered on frontend.type
== "mobile"
// (dt.smartscape.frontend). SaaS 1.344 (07/27/2026, staged tenant rollout
from 07/29/2026)
// makes Smartscape the primary surface for the Digital Experience apps;
verify it has
// reached your tenant. The classic table below still works and remains a real
fallback.
// FALLBACK (classic surface -- still functional):
// fetch dt.entity.mobile_application
// | fields entity.name, id, lifetime, tags
// | limit 10
```

Summary

In this notebook you learned how to:

- **Create a mobile app configuration** in Dynatrace and obtain the Application ID and Beacon URL
- **Install the Dynatrace iOS SDK** via Swift Package Manager or CocoaPods
- **Configure** `Info.plist` for auto-start, crash reporting, and beacon communication
- **Leverage auto-instrumentation** for UIKit view controllers, navigation, and network requests
- **Instrument SwiftUI controls** at build time with the Dynatrace SwiftUI instrumentor (`DTSwiftInstrumentor`)
- **Manually start the SDK** for deferred initialization or user-consent scenarios

- **Verify data flow** using DQL queries against the mobile `FRONTEND` inventory and `user.events`

Next Steps

Continue to **MOBL-03: Android SDK Setup (Kotlin & Jetpack Compose)** for the Android counterpart. Custom user actions are covered in **MOBL-05: User Action Tracking**, user tagging in **MOBL-09: Session Properties & Data Privacy**, and business events in **MOBL-12: Advanced Instrumentation & Optimization**.

References

- Dynatrace Mobile Monitoring Documentation
- iOS SDK Integration Guide
- SwiftUI Instrumentation
- OneAgent SDK for iOS (DT docs)
- Mobile App Configuration Settings

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.